NEXUS · COMMUNITY GUIDE

PCIe DMA Firmware Engineering

A practical engineering reference for building 1:1 device emulation firmware on Artix-7 FPGAs. Covers the donor concept, configuration space, capability chains, BDF stability, MSI cadence, IOMMU realities, AHCI/SATA emulation, the Windows storage stack walls you will hit, Vivado build reality, and live verification.

Published by Nexus for the community.
Vendor-agnostic. No proprietary code. No anti-cheat specifics. Provided as engineering reference.

Disclaimer

This document is provided for educational and research purposes only.

The information in this guide describes general engineering principles of PCIe device emulation on FPGA hardware. It is intended for legitimate purposes including, without limitation:

- Hardware reverse-engineering research on devices the reader legally owns or has authorisation to inspect.
- Custom FPGA-based test instrumentation and fixtures for development workflows.
- Hardware-in-the-loop simulation of PCIe peripherals for driver development.
- Security research conducted with explicit authorisation from the system owner.
- Academic study of bus-level protocols, configuration space semantics, and storage stack behaviour.

No warranty

The author makes no warranty as to the accuracy, completeness, fitness for any particular purpose, currency, or absence of errors in this material. The information is provided "as is". The reader uses it entirely at their own risk.

No liability

The author accepts no liability for any direct, indirect, incidental, special, consequential, or exemplary damages of any kind arising from or in connection with the reader's use of the techniques, code, scripts, register values, command sequences, or other content described in this document. This includes (without limitation) damage to hardware, loss of data, system instability, voided manufacturer warranties, regulatory issues, breach of contract, or any third-party claim.

Reader responsibilities

The reader is solely responsible for ensuring that any use of the techniques described in this document complies with:

- All applicable local, national, and international laws and regulations.
- The terms of service of any software the reader interacts with.
- The end-user licence agreements and warranties of any hardware the reader modifies, inspects, or emulates.
- The policies and authorisation requirements of any system, network, or platform on which the reader's firmware is loaded.

The author does not endorse and is not responsible for any use of this material in violation of any of the above.

Distribution

Redistribution and modification of this document for non-commercial purposes is permitted provided that this Disclaimer remains attached, unmodified, to any derivative work. Commercial redistribution requires the prior written consent of the author.

By reading further you acknowledge that you have read, understood, and accept the terms of this Disclaimer.

Contents

01 Introduction & Scope

02 Hardware Platform and Toolchain

03 The Donor Concept

04 Configuration Space

05 Capability Chains

06 BDF Stability and the Heartbeat Tell

07 MSI Behavior

08 IOMMU Realities

09 Storage Class Emulation: AHCI / SATA

10 ATA Command Engine Architecture

11 Disk Image, Filesystem, and the partmgr Wall

12 Writes and Persistence

13 SystemVerilog Pitfalls

14 Vivado Build Reality

15 Per-Board Identity (DNA Binding)

16 Flashing and Deployment

17 Live Verification Probes

Quick Start. Your First Build, Step by Step

Appendix A. Common Errors and Diagnostics

Appendix B. Reading List

Appendix C. Glossary

1. Introduction & Scope

This guide is about building PCIe DMA firmware that presents itself to a Windows host as a real, off-the-shelf PCIe device, end to end.

Not just matching a vendor and device ID, but matching the full surface: configuration space byte-by-byte, capability chains, link behavior, interrupt cadence, class-specific register semantics, the device's storage stack interaction, the strict validation behavior of Windows kernel drivers, and the per-board identity that makes one card distinguishable from another.

The platform target throughout is the Xilinx Artix-7 family in CaptainDMA-class boards (XC7A35T, XC7A75T, XC7A100T). The host operating system is Windows 10 / 11 x64. The reference device class used as the worked example is SATA / AHCI. The concepts generalize to any PCIe device class; the storage stack is the most demanding case because the validation surface above the device is the largest.

What this guide is

- A practical engineering reference for people who already understand HDL, Vivado, and PCIe at the spec level, and who have built or modified firmware before.
- A consolidation of the non-obvious problems you only learn by hitting them: silent multi-driver synthesis bugs, transient BDF events, partmgr's hardcoded LBA 2 read, fastfat's directory-corruption rules, the difference between IOMMU presence and IOMMU enforcement.
- Vendor-agnostic. No specific donor card, board serial, or production firmware version is named.

What this guide is not

- It is not a tutorial on PCIe fundamentals. If you do not know what a TLP, BAR, or capability pointer is, read the PCIe Base Specification first.
- It is not an anti-cheat circumvention guide. It does not describe how to evade detection by any specific anti-cheat product. It describes what real silicon looks like from the host's perspective, which is the only thing your firmware should be aiming at.
- It is not a copy of anyone's production source. Every code-shaped block in this guide is illustrative and minimal, not lifted from a working build.

How to read it

The chapters are roughly in build order. Chapters 2 through 8 are platform and protocol. Chapters 9 through 12 are class-specific (the storage path). Chapters 13 through 17 are toolchain, deployment, and verification. The appendices are diagnostic. If you are debugging a specific symptom, jump to Appendix A first.

> **One framing rule that saves time:** a "1:1" firmware is not a firmware that looks like a particular product. It is a firmware where every byte and every behavior an inspector can read matches the equivalent byte and behavior on real silicon. The right reference is always the actual chip on a bench, not a community-shared dump of the same model.

Quick Start: Your First Build, Step by Step

If you have never built DMA firmware before, do this end to end before reading the rest of the guide. It teaches the loop (edit, build, flash, observe) and lets you find broken toolchain before broken HDL.

Q.1 Hardware checklist

You need two PCs, one FPGA board, two cables, and one (ideally two) donor cards. See Chapter 2.3 for the full PC1 / PC2 topology explanation.

Item	What it does
PC1 (Host PC) Windows 10/11 x64, free PCIe x1 or x4 slot.	The machine your firmware presents itself to. The FPGA board goes into PC1's PCIe slot. Windows on PC1 enumerates the device, binds drivers, runs the host-side probes from Chapter 17. PC1 also hosts the donor card temporarily during capture (Chapter 3).
PC2 (Operator PC) Windows or Linux, modern desktop or laptop.	Where you edit HDL, run Vivado, version-control sources, flash the FPGA via NexusForge, and run any user-mode FT601 tools. PC2 connects to the FPGA only via the USB-C cable.
FPGA board CaptainDMA-class Artix-7: 35T, 75T, or 100T.	The hardware your firmware runs on. Goes into PC1's PCIe slot. 75T is the standard target. Has a USB-C port for FT601 / JTAG-over-USB that connects to PC2.
USB-C cable (FPGA board to PC2)	Carries JTAG-over-USB (for flashing via NexusForge) and FT601 bulk endpoints (for user-mode tools). One physical cable, both functions multiplexed by the FPGA's USB controller.
Donor card A SATA AHCI controller, ideally two identical ones.	The real silicon you are emulating. Installed into PC1's PCIe slot temporarily during Chapter 3 capture. See Chapter 3.2 for what to buy and what to avoid.
JTAG cable (optional, only if your board does not expose JTAG-over-USB)	Some boards expose JTAG only through a dedicated header; these need a CH347 or FT2232H pod plugged into PC2. CaptainDMA-class boards use the USB-C cable for both, so this is usually not needed.

Useful but optional: a PCIe extension riser cable for safe insertion / removal of the FPGA from PC1 during testing, and a power strip with a hardware switch on PC1 so you can do clean cold cycles between flashes.

> **Can I use one PC for both roles?** Yes, but it is painful in practice. Every reflash needs PC1 to fully cold-cycle, which means closing Vivado, shutting down Windows, power-off, power-on, logging in again, then re-running probes. With two physically separate PCs you can keep Vivado open on PC2 while PC1 cold-cycles in seconds. For Day 1 you can absolutely use one PC; for sustained development, two is much faster.

Q.2 Software setup on PC2 (Operator PC)

PC2 is where you build firmware and flash the FPGA. Set it up first.

1. **Install Vivado 2022.2** (or the version your board project specifies). Use the Webpack edition, no licence required. Pick a short install path (`C:\Xilinx\Vivado\2022.2`) to avoid the Windows MAX_PATH trap later.
2. **Install Python 3.11** from python.org. Add to PATH. Avoid Python 3.14 for any tooling that needs binary wheels.
3. **Install NexusForge** from github.com/NexusForge-dev/NexusForge. Single self-contained `.exe` . The CH347 driver is bundled and offered on first detect when you plug in the FPGA via USB-C.
4. **Install a Git client** (Git for Windows is fine). You will be cloning HDL repositories.

Q.3 Software setup on PC1 (Host PC)

PC1 is the machine Windows runs on, with the FPGA card in its PCIe slot. This is the only PC that sees the firmware as a PCIe device.

1. **Disable Windows Fast Startup** (Control Panel, Power Options, Choose what the power buttons do, uncheck "Turn on fast startup"). Without this, "Shut down" leaves the FPGA partly powered and the new bitstream may not load on the next boot.
2. **Verify no virtualisation features are running** that would activate the IOMMU enforcement stack. From an admin PowerShell on PC1:

   ```
   Get-CimInstance Win32_ComputerSystem | Select HypervisorPresent
   Get-CimInstance Win32_DeviceGuard -Namespace root\Microsoft\Windows\DeviceGuard `
       | Select VirtualizationBasedSecurityStatus, SecurityServicesRunning
   ```

   All three values should be `False` / `0` / `{0}` . If any of them are not, follow the recovery in Chapter 8.5 (full shutdown then power-on, not restart).
3. **Install RWEverything** on PC1 for raw configuration-space inspection of either the donor or the FPGA.
4. **Install Arbor** on PC1 for the structured cfg-space inspector. Arbor is distributed through the Nexus Discord, see Chapter 3.5 for the access procedure.

You do not need to install Vivado or NexusForge on PC1. Those live on PC2.

Q.4 Day 1 sequence

Do this in order. Each step verifies the previous step worked.

1. **On PC1: install the donor card in PC1's PCIe slot. Boot PC1 and confirm Windows binds the expected driver.** If Windows shows the wrong driver (or no driver), your VID/DID/REV is not what you expected. Stop and resolve before continuing. If you bought the wrong card, return it.
2. **On PC1: capture the donor's configuration space with Arbor (or boot a Linux live USB on PC1 and run `lspci -xxxx -vvv` , see Chapter 3.5 and 3.6).** Save this as `donor_cfg_baseline.txt` . It is your ground truth.
3. **On PC1: run the host-side probes from Chapter 17 against the donor.** Save the outputs as `donor_probes_baseline.txt` . This tells you what your firmware's emulation needs to look like to

Windows.

4. **Shut down PC1. Remove the donor card. Install the FPGA board in PC1's PCIe slot. Connect the FPGA's USB-C cable to PC2. Power on PC1.** Confirm the FPGA enumerates on PC1 as some PCI device (whatever its current identity is). The fact that Windows sees anything at all tells you the PCIe slot, the board, and the link are healthy.
5. **On PC2: clone `pcileech-fpga` from github.com/ufrisk/pcileech-fpga** (see Chapter 2.5 for context on this and other starter bases). Open the project variant whose name matches your board (e.g. `pcileech_75t484_x1` for a 75T x1 board) in Vivado. Build with no modifications. Confirm Vivado produces a `.bin` and that timing closes (WNS / WHS both positive in the implementation report).
6. **On PC2: launch NexusForge, click Detect.** The FPGA should appear (via the USB-C cable). Pick the `.bin` from step 5 and click Flash. Wait for the verify pass.
7. **Cold-cycle PC1.** Full shutdown (not restart), then power on. The FPGA loads the new bitstream during PC1's POST. Confirm Windows on PC1 now sees the device the template advertises.
8. **On PC2: modify exactly one byte of the configuration-space COE** (a safe one, like one of the Subsystem Vendor ID bytes if the template lets you change it). Rebuild. Wipe the IP cache (Chapter 14.2). Reflash via NexusForge. Cold-cycle PC1. Confirm Windows on PC1 now sees the new SVID byte.

If step 8 worked, your end-to-end edit-build-flash-observe loop is operational. From here, the rest of the guide is the actual engineering work.

Q.5 When something doesn't work, look here first

Symptom	Where to start
Vivado errors with cryptic `.Xil` path messages	MAX_PATH overflow. Shorten the project path. Chapter 2.4.
Build succeeds but `.bin` is byte-identical after a COE change	IP cache. Chapter 14.2.
FPGA enumerates but Windows shows "Unknown device" or no driver	VID/DID/Class mismatch. Capture the FPGA's cfg-space with RWEverything and diff against the donor.
FPGA enumerates correctly but disappears under load	BSOD, watchdog, or surprise removal. Check Event Viewer (System log) for storahci, fastfat, or HAL events. Match to Appendix A.
HAL Event 15 in System log	BDF transient or active IOMMU enforcement. Chapter 6 and 8.
fastfat Event 50 status 0xC000000E	PRDBC = 0 on a write completion. Chapter 12 and Appendix A.5.
Disk shows up but `IsReadOnly` stuck True	partmgr LBA 2 cache. Chapter 11.
FPGA vanishes from Device Manager on rescan	Hard IP wedged. Add watchdog timers (Appendix A.6).

Q.6 Mindset

DMA firmware development looks like software but behaves like hardware. Three habits save the most time:

- **Change one thing at a time.** If your build was working and now it isn't, the suspect is whatever you changed last, not the obscure thing you read about in a forum.
- **Keep every working `.bin` file** with a version tag and a one-line note about what's different. You will need to bisect failures across versions. Disk space is cheap.
- **Trust the donor more than your assumptions.** When in doubt about what a register should look like, re-capture from the donor under the same conditions. The real chip is always the source of truth.

When you are completely stuck for a day on something that "should work", the answer is almost always one of: an IP cache that wasn't wiped, a forgotten power cycle, a silent multi-driver bug, or a partmgr / fastfat strictness rule. Walk through that list before deeper rabbit holes.

2. Hardware Platform and Toolchain

Artix-7 + Vivado is the standard target. Most failures here come from mis-managed paths, stale IP caches, and Python version drift.

2.1 Why Artix-7

Artix-7 (7-Series) parts give you a 7-Series Integrated Block for PCI Express (the "hard IP") with Gen1/Gen2 x1, x2, x4, x8 support, integrated PHY, and a free Vivado WebPack toolchain. The relevant device sizes in commodity DMA boards are:

Part	Logic Cells	BRAM (RAMB36)	DSP	Notes
XC7A35T	33,280	50 (1.8 Mb)	90	Tight for storage emulation with rich disk images. Fine for thin DMA cores.
XC7A75T	75,520	105 (3.8 Mb)	180	Standard target. BRAM budget allows realistic disk slot maps.
XC7A100T	101,440	135 (4.9 Mb)	240	Headroom for write coalescing, large slot maps, or extra protocol layers.

2.2 Toolchain

Vivado 2022.2 is the most common build target. Newer versions work but you'll need to handle IP migration warnings. Older versions are missing fixes for some PCIe Hard IP corner cases.

- **Vivado:** install path under `C:\Xilinx\Vivado\<version>`. Add the `bin` folder to PATH or invoke directly.
- **Python:** needed for image generation scripts. Invoke as `py` on Windows. Avoid Python 3.14 for tooling that depends on older binary wheels; 3.11 is the safe baseline.
- **JTAG flasher (recommended): NexusForge**, a Windows GUI that wraps the whole flashing pipeline (board detection, DNA read, driver bundle, flash write, verify) with embedded OpenOCD + CH347 driver. Get it from github.com/NexusForge-dev/NexusForge. Recommended over raw OpenOCD or vendor CLIs for first-time users, it removes most of the "which interface config file does my board want" complexity.
- **JTAG flasher (CLI alternative):** OpenOCD with a CH347 or FT2232H adapter if you prefer a scripted toolchain.
- **Host inspection on Windows:** Device Manager, `Get-PnpDevice`, `Get-PhysicalDisk`, `RWEverything` for raw cfg-space dumps, and an arbitrary PCI bus analyzer or trace tool when you need silicon ground truth.

2.3 The two-PC topology

A realistic DMA firmware setup involves **two physical PCs**, each with a distinct role. They are connected to the FPGA board through different physical paths.

Role	What it is	How it connects to the FPGA
PC1 (Host PC)	The Windows machine that runs the operating system your firmware presents itself to. Windows enumerates the FPGA card on its PCIe bus, binds the matching driver, mounts any storage, and runs all the host-side tools you use to validate the emulation (Device Manager, PowerShell probes, RWEverything, Arbor, Event Viewer).	The FPGA board is **physically installed in PC1's PCIe slot**. PCIe lanes carry configuration cycles, TLPs, MSI, and DMA between the FPGA and PC1's CPU/RAM.
PC2 (Operator PC)	The Windows or Linux machine where you run Vivado, edit HDL, version-control sources, flash the FPGA via JTAG, and run any user-mode tools that drive the FPGA (NexusForge, JTAG flashers, FT601 user-mode tools). PC2 never sees the firmware as a PCIe device, only as a USB peripheral.	The FPGA board's **USB-C / FT601 port is cabled to PC2's USB**. This USB path is separate from PCIe and is used only for board control: JTAG-over-USB for flashing, FT601 bulk endpoints for user-mode tools.

> **Convention used in this guide:**
>
> - **PC1 / Host PC** = where the FPGA card is plugged into a PCIe slot. This is where Windows enumerates your emulated device.
> - **PC2 / Operator PC** = where the FPGA's USB cable terminates. This is where you build, flash, and run user-mode tools.
> - **The device** = the firmware running on the FPGA, viewed from PC1's perspective.
>
> The two PCs can be (and often are) the same physical machine if you are willing to power-cycle and switch monitors / keyboards between roles. Most active development is easier with two physically separate PCs because PC1 must be cold-rebooted every time the firmware is reflashed.

2.4 What you do on each PC

Activity	PC
Edit HDL, run Vivado, version-control sources	PC2
Build `.bit` and convert to `.bin`	PC2
Run NexusForge or OpenOCD to flash the SPI	PC2 (the USB cable is here)
Power-cycle the FPGA so the new firmware loads	PC1 (where it draws PCIe power)
Run Device Manager, `Get-PnpDevice`, `Get-PhysicalDisk`	PC1
Capture cfg-space with RWEverything or Arbor	PC1
Read Event Viewer / fastfat / storahci event logs	PC1
Run user-mode FT601 tools (DMA controllers, etc.) against the FPGA	PC2 (via USB)

Build artifacts transfer from PC2 to PC1 (and back) via shared network drive, USB stick, PSRemoting, or any file-share mechanism you prefer.

The FPGA only loads new firmware on power cycle or PROG_B assertion, reflashing without a power cycle does nothing visible. The power cycle happens on PC1 because that is where the FPGA is drawing its slot power.

2.5 Starter codebases

Do not start from a blank Vivado project. Build on an existing open-source PCIe HDL base. The realistic options for a beginner working on the Artix-7 family:

Codebase	What it gives you
pcileech-fpga github.com/ufrisk/pcileech-fpga	The de facto community PCIe HDL base for Artix-7 DMA boards. SystemVerilog source, includes pre-configured projects for CaptainDMA / PCIeSquirrel / Enigma X1 / ZDMA / Screamer and other common board families. Wraps the 7-Series PCIe Hard IP, exposes a BAR-mapped engine, and provides reference configuration-space templates. This is what the Quick Start refers to in step 5.
PCILeechFWGenerator github.com/voltcyclone/PCILeechFWGenerator	Python toolchain that takes a captured donor configuration space and auto-generates the cfg-space COE, the writemask COE, and the per-board source customisations on top of pcileech-fpga. Saves you the hand-templating step. Useful once you understand what those templates do (Chapters 4 and 5).
verilog-pcie github.com/alexforencich/verilog-pcie	Alex Forencich's modern PCIe AXI-Stream IP collection. Cleaner abstraction than the legacy bases, better-suited to high-throughput applications. Worth reading for reference. Less convenient as a starter for 1:1 device emulation because you build the configuration-space side yourself.
litepcie github.com/enjoy-digital/litepcie	LiteX / Migen-based PCIe IP, Python-driven generation. Excellent for prototyping if you are comfortable with Migen. Less common in the DMA community.
Xilinx XAPP1052	Xilinx's own PCIe DMA reference design. Older, but a clean teaching reference for the TX / RX state machine if you want to understand the hard-IP interface before layering anything on top.

For Quick Start (first build), clone **pcileech-fpga**, pick the variant whose top-level project matches your board (e.g. `pcileech_75t484_x1` for a 75T board with x1 connector), open it in Vivado, and run synthesis with no modifications. The bitstream that comes out is functionally valid as-is; you change identity bytes and class-specific behaviour from there.

2.6 Windows path-length trap

Windows' classic 260-character MAX_PATH limit will silently kill Vivado builds with cryptic `.Xil` file errors when the project name plus a deep auto-generated path exceeds 260 characters. Use a short Vivado project name (4–6 characters) and put the project root at a shallow path like `D:\proj\xxx`. Do not rely on long-path support being enabled.

3. The Donor Concept

Choosing the right donor class is the single highest-leverage decision a starter makes. Picking the wrong class can waste months. This is the chapter to read twice.

3.1 What “donor” means

A donor is the real, physical PCIe device whose identity and behavior your firmware will emulate. You need a donor that:

- Is the exact same revision (REV byte) of the same model. Stepping changes alter capability layouts in non-obvious ways.
- You can physically access for cfg-space dumps, link behavior captures, and ideally protocol-level traces.
- You have used long enough in a real system to know how it behaves at link state transitions (warm reset, hot reset, ASPM L1 exit, FLR if the device supports it, PME).

3.2 Choosing a donor class

Not all PCIe device classes are equal. Some are simple, well-spec’d, and have predictable host driver behavior. Others have validation surfaces so large that emulating them is a multi-year project on its own. Pick from the green column.

Recommended (start here)	Avoid (deep water)
SATA / AHCI controllers, the standard first-project donor. Small register surface, well-defined AHCI 1.3.1 spec, predictable `storahci.sys` driver behavior. Class code 0x010601.	**NICs (Ethernet, Wi-Fi)**, vendor driver checks MAC address EEPROM, link partner negotiation, statistics counters polled continuously, PHY register access, Wake-on-LAN, magic packet handling. Months of work just to satisfy the driver, before you even think about traffic.
USB-to-SATA bridges exposing AHCI, same as SATA controllers, often even simpler.	**GPUs**, massive BAR surface, complex MMIO register set, vendor driver expects detailed PCIe-side behavior. Effectively impossible for starter projects.
Simple PCIe-to-PCI bridges (non-transparent bridges from the early 2010s), minimal cfg-space, no class-driver surface to emulate.	**NVMe controllers**, admin submission queue, multiple I/O queues, doorbell registers, complex state machine. Spec is well-documented but the surface is huge.
SD / MMC card readers, small register set, simple protocol.	**Smart NICs** (datacenter-class), SR-IOV, virtualization features, vendor firmware loaded at init. Worst-case.
	RAID controllers, proprietary firmware, vendor-specific driver, often emit verification probes the driver expects specific responses to.
	Wi-Fi / Bluetooth combo cards, firmware uploaded over PCIe at driver init. You’d need to emulate the firmware load path.

> **Why SATA AHCI is the canonical first donor:** the host driver (`storahci.sys`) is in every Windows install, well-understood by the community, and the AHCI 1.3.1 + ATA Command Set specs together describe its entire host-visible behavior. The register surface is roughly 0x80 bytes of Generic Host Control + 0x80 per port. You can hold the whole thing in your head. That is true of essentially no other modern PCIe class.

3.3 Sourcing a donor

For starter projects, you want a cheap, available, well-documented chip. Practical tips:

- **Buy at least two identical cards.** One for daily work (re-dumps, experiments, even brick-risk JTAG probing), one as the untouched reference. The cost difference is small; the time saved when your primary donor dies is large.
- **Match the exact PCI VID / DID / REV before purchase.** eBay sellers don't reliably list these. Ask the seller for a photo of the device in Windows Device Manager (Properties → Details → Hardware Ids), or a `lspci -nn` output. The hardware ID looks like `PCI\VEN_XXXX&DEV_YYYY&REV_ZZ`; all three fields must match your build target.
- **Used market is fine.** Low-end SATA AHCI controllers from the 2010–2018 era are typically $5–30 on the used market. Older = more spec/community info, cheaper, and the underlying silicon often hasn't changed across multiple board vendors using the same chip.
- **Avoid "RAID"-branded cards** unless the same chip is also sold in a non-RAID variant. RAID cards may have proprietary firmware that changes the cfg-space layout vs the AHCI variant.
- **Avoid the newest generation.** Less spec data published, less community knowledge, harder to find a second card if yours fails. The 5-year-old part is usually a better donor than the brand-new part.

3.4 Why the shipped templates are not enough

Many DMA toolchains ship pre-baked donor captures. Treat these as starting points only. They have well-known tells:

- Junk bytes in reserved configuration-space regions that real silicon never sets.
- Phantom capability headers (a capability ID and pointer at an offset where the real device has none).
- Expansion ROM Base Address Register populated when the real device exposes no expansion ROM.
- Reserved bits set in standard capability registers that the real chip leaves clear.
- Bytes in the extended region (offsets 0x100–0xFFF) that look copied from a different device.

The right reference is always the actual chip on your bench, not a community-shared dump of the same model. Two seemingly identical cards from different production runs can differ in subtle ways (subsystem IDs, capability bit defaults, fused-but-disabled features).

3.5 Capture workflow on Windows (Arbor)

Arbor is the recommended Windows-side cfg-space inspector. It loads a kernel-mode helper driver, enumerates the PCIe bus tree, and provides a structured, interpretive view of every capability. The Nexus community redistributes it through Discord so the version you get is known-good and matched against the rest of this guide's workflow.

HOW TO GET ARBOR

1. Join the Nexus Discord: discord.gg/y49jPf8T5z.
2. Complete the server verification.
3. Open the `#download-arbor` channel. The download link and install instructions are pinned there.

CAPTURE PROCEDURE

1. Install the donor card in PC1's PCIe slot. Disable anything that might touch cfg-space at runtime on PC1 (vendor utilities, monitoring tools, hardware-info apps like CPU-Z, HWiNFO).
2. Install Arbor on PC1. Accept the kernel helper driver prompt the first time you launch.
3. Launch Arbor as Administrator.
4. In the device tree, navigate to your donor by BDF (Bus:Device.Function). Right-click → **Get Config** to capture the full 4 KB of cfg-space.
5. Use the interpretive views (Header, Capabilities, Extended Capabilities) to read what every bit means, far more useful than a hex dump for understanding which fields are set.
6. Export the capture as text or XML for diffing across captures.

Arbor's interpretive view is the biggest practical advantage over raw `RWEverything`: it decodes each capability into its named fields so you can immediately see, for example, that the donor has Aux_PME = 0 in Device Control even though many community dumps have it set.

3.6 Capture workflow on Linux

If you don't want to install Arbor (or want a cross-check), a Linux live USB is the simplest alternative. Any modern distro will do (Ubuntu, Mint, Fedora). No need for a dedicated Linux machine, boot the live USB on the donor's host.

```
# Identify the donor's BDF (Bus:Device.Function):
sudo lspci -nn

# Verbose decoded output (capability-by-capability):
sudo lspci -vvv -s 06:00.0          # replace with your donor's BDF

# Raw 4 KB hex dump:
sudo lspci -xxxx -s 06:00.0

# Save to file for diffing:
sudo lspci -xxxx -s 06:00.0 > donor_cfg_idle.txt

# Alternative: raw byte file
sudo cat /sys/bus/pci/devices/0000:06:00.0/config | xxd > donor_cfg_idle_xxd.txt
```

The Linux output from `lspci -vvv` decodes every capability the way Arbor does, just as plain text. The `-xxxx` flag dumps all 4 KB (extended cfg-space included); `-xxx` dumps only the first 256 bytes; `-xx` only the first 64. Use `-xxxx` for donor capture.

3.7 Capture under multiple conditions

One capture is not enough. Several fields are dynamic and only reveal their nature when you observe them changing. Take captures in these states and diff them:

- **Idle, just after cold boot.** The most static baseline.
- **Idle, after 5+ minutes of uptime.** Some chips update internal counters even at idle; rare but worth checking.
- **Under sustained traffic.** Status register bits, link state counters, and a few capability-specific bits change.
- **After warm reboot (without power cycle).** Power Management state (PMCSR PowerState) and Aux_PME may differ.
- **After cold power cycle.** The strongest reset; everything goes back to silicon defaults.
- **After suspend/resume (S3) if your platform supports it.** Reveals PM-related dynamic bits.

Diff all the captures. Anything that changed between any two is dynamic and must be modeled in your firmware (or, for fields the host writes, simply left to host-side updates of the shadow). Anything that's stable across every capture is what your COE file should hold.

3.8 Capturing host-visible state for comparison

While the donor is installed and running on the Windows host, you can also capture how the host's storage stack sees it. These captures become your ground truth for what your firmware's class behavior should look like:

```
Get-PhysicalDisk | Where-Object { $_.BusType -eq 'SATA' } | Format-List *
Get-Disk | Format-List Number, PartitionStyle, IsReadOnly, IsOffline, UniqueId
Get-Partition | Format-List DiskNumber, PartitionNumber, DriveLetter, Type, IsHidden, IsActive
Get-CimInstance -Namespace root\wmi -ClassName MSStorageDriver_FailurePredictData |
    Format-List *
```

Save these outputs. When you flash your firmware and run the same probes against the FPGA-emulated disk (Chapter 17), they should match field-by-field.

3.9 Do's and don'ts at a glance

Do	Don't
Buy 2+ of the exact same donor (same VID/DID/REV).	Pick "something similar" and hope the cfg-space is close enough.
Capture from your own donor in multiple conditions.	Trust a community-shared dump as your only source.
Verify the rev byte matches what your firmware is targeting.	Assume revs of the same model are interchangeable.
Start with a SATA AHCI controller class for your first project.	Start with a NIC, GPU, NVMe, or Wi-Fi card.

Use Arbor on Windows or `lspci -xxxx -vvv` on Linux.	Rely only on Device Manager, it shows the host's interpretation, not the raw bytes.
Diff your captures and document which bytes are dynamic.	Bake the "current" dump into the COE without distinguishing static from dynamic.
Keep the donor card around, you'll need it for re-dumping when you hit a wall.	Sell or scrap the donor once your firmware "works", some walls only show up after months.

3.10 What gets into the COE

The configuration space COE file (a Vivado RAM init file) holds the *initial* contents of the cfg-space shadow RAM. Host writes to writable fields (Command register, Cache Line Size, capability control registers, MSI message data, etc.) update the live shadow at runtime. The COE only sets what the host reads at link-up or after PERST# deassertion.

Rule of thumb: any byte not in your captured idle dump, not a known dynamic field, and not host-writable should be zero. Donor templates that pre-populate reserved regions with anything other than zero are wrong.

4. Configuration Space

4 KiB per function. Every byte is potentially inspected. Reserved means zero, not "whatever the donor template left there".

4.1 Layout overview

PCIe configuration space is 4 KiB per function, divided into:

- **0x000–0x03F, PCI-compatible Type 0 header.** VID, DID, Command, Status, Class code, Revision, Cache Line Size, BARs, Subsystem IDs, Cap Pointer, Interrupt Line/Pin.
- **0x040–0x0FF, legacy capability region.** Capability structures linked from the Cap Pointer at 0x34. Each capability has an ID byte, a next-pointer byte, and a payload. Typical capabilities here: Power Management (0x01), MSI (0x05), PCIe Capability (0x10), Vendor-Specific (0x09), MSI-X (0x11), SATA-specific (0x12 if present).
- **0x100–0xFFF, PCIe extended capability region.** 4-byte headers with capability ID, version, and 12-bit next-pointer. Things like AER (0x0001), Virtual Channel (0x0002), Device Serial Number (0x0003), Power Budgeting, ACS (0x000D), ATS (0x000F), SRIOV (0x0010), L1 PM Substates, Resizable BAR.

4.2 Fields that look static but aren't

Offset	Field	Why dynamic
0x04	Command	Bits 0–2 (IO/MEM/BME) are host-written at enumeration. Bit 10 (INTx Disable) toggles with driver state.
0x06	Status	Capabilities List bit static. Detected Parity, Signaled System Error, Master Abort, Target Abort, INTx Status are runtime.
0x0C	Cache Line Size, Latency Timer	Host writes Cache Line Size during enumeration. Latency Timer is meaningless for PCIe but the host may write it.
0x10–0x24	BARs	Lower bits encode type and size at reset; host writes upper bits to assign address.
0x3C	Interrupt Line	BIOS-written at boot. Interrupt Pin is fixed.

4.3 The "tells" pass

When you have your donor capture and your firmware's current cfg-space side-by-side, walk every offset and check:

1. **VID, DID, SVID, SSID, Class, RevID** exactly match. Class code is 3 bytes; the revision is at 0x08 in the LSB of the 32-bit dword.
2. **BAR size and type** match: I/O vs Memory, 32-bit vs 64-bit, prefetchable vs not, size determined by the bottom bits.
3. **Cap Pointer at 0x34** matches the donor's first capability offset.

4. **Reserved bytes** between capability structures are zero. If the donor template has nonzero bytes in 0x40–0x4F or 0x60–0x77 etc. where no capability lives, those are tells.
5. **Expansion ROM BAR at 0x30:** if the donor reads as 0x00000000 (no ROM exposed), your firmware must also return 0x00000000. A non-zero Expansion ROM BAR with no actual ROM behind it is a tell that any thorough scan will catch.
6. **Extended space 0x100–0xFFF:** zero by default. Only the capabilities the donor actually exposes should be populated. Anything else is a tell.

> **Cache pitfall:** Vivado caches synthesized IP outputs aggressively. When you change a COE file (the BRAM init for cfg-space), Vivado will happily reuse the old synthesized output unless you wipe the IP run directory and force regeneration. Always delete the cached IP outputs before rebuilding after a COE change, and verify the bin size or hash differs from the previous build.

4.4 Writemask

The cfg-space writemask COE controls which bits the host can write through the shadow path. The donor's writemask is harder to capture directly; you derive it from the spec for each capability plus observation of which bits change after host operations. Get the writemask wrong in either direction and you have a tell: too permissive lets the host change identity bits; too restrictive makes host writes silently fail and the driver behavior diverges.

5. Capability Chains

Match what the donor chip exposes. Inherited templates often carry phantom capabilities (DSN, AER, MSI-X) that low-end chips don't have.

5.1 Legacy chain

Cap Pointer at 0x34 points to the first capability. Each capability has a one-byte ID, a one-byte next-pointer, and a payload. The chain terminates when next-pointer = 0x00.

A typical real SATA AHCI controller's legacy chain looks like:

```
0x50 [MSI, ID=0x05]    -> next=0x78
0x78 [PM,  ID=0x01]    -> next=0x80
0x80 [PCIe Cap, ID=0x10] -> next=0x00 (terminator)
```

That's three capabilities, totaling 8 + 8 + 60 = 76 bytes of structure. Everything else in 0x40–0xFF is zero. If your donor template has bytes at 0x68, 0xEC, 0xF0–0xFF, those are tells unless the real chip actually has capabilities there.

5.2 Extended chain

Starts at 0x100. Same idea: 4-byte header (16-bit capability ID, 4-bit version, 12-bit next-pointer offset). Terminator = next-pointer 0x000.

Most low-end PCIe endpoints have *nothing* in the extended region except possibly Virtual Channel. Things you may incorrectly inherit from a donor template that the real chip doesn't have:

- **Device Serial Number (DSN, 0x0003)**, many low-end devices don't expose this. If your firmware does, that's a per-board fingerprint anyone can read.
- **AER (0x0001)**, almost no low-end SATA / NIC / etc. exposes Advanced Error Reporting.
- **ATS, ACS, SRIOV**, mostly server/datacenter parts. Consumer chips don't have them.
- **L1 PM Substates**, modern, common on mobile / laptop chips; less common on older controllers.
- **MSI-X (0x11 in legacy space)**, many low-end chips offer only legacy MSI. A phantom MSI-X capability with no actual table is a strong tell.

5.3 Power Management capability

The PM cap has two registers after the header:

- **PMC (Power Management Capabilities)** at +0x02 of the cap. Encodes version, AUX current, D1/D2/D3hot/D3cold support, PME support. Static for a given chip.
- **PMCSR (Power Management Control/Status)** at +0x04. PowerState bits (0–1) are host-writable; the host transitions the function through power states.

The PMCSR PowerState value is dynamic. Capturing it during the donor dump while the device is in D0 versus D3hot will give you different values. Bake PowerState = D0 by default; the host will write D3hot

when it wants to suspend.

5.4 PCIe Capability structure

Anchored at 0x80 in most layouts. Contains:

- PCIe Capabilities (capability list version, device type, slot bit, interrupt message number), static.
- Device Capabilities, static, encodes max payload size, phantom function support, extended tag support, endpoint L0s/L1 acceptable latency, etc.
- Device Control, host-writable. Bit 11 (Aux Power PM Enable) is frequently set by donor templates but unset on the real chip. Verify against the donor capture and clear if needed.
- Device Status, mostly read-only error bits, plus the Transactions Pending bit.
- Link Capabilities, static, encodes max speed, max width, ASPM support, L0s/L1 exit latency.
- Link Control / Status, partly host-writable (Common Clock Configuration bit, ASPM control). Status (current link speed, width, training) is dynamic from the link layer.
- Device Capabilities 2, Control 2, Status 2, v2-only fields. Latency Tolerance Reporting, OBFF, etc.
- Link Capabilities 2 / Control 2 / Status 2, Target Link Speed in Control 2 may be set by the host to throttle.

5.5 The Aux_PME tell

Device Control bit 11 (Aux Power PM Enable) is enabled by the host only when the device declared PME support via AUX in PMC. Many donor templates ship with bit 11 already set in the static COE, which is wrong: the host writes it, the device doesn't initialize it. If your donor capture shows bit 11 clear at link-up, your COE must hold it clear.

6. BDF Stability and the Heartbeat Tell

Two classic tells in DMA firmware: a BDF that flickers under link transients, and a synthetic MSI heartbeat that no real chip would emit during idle.

6.1 What BDF is

Bus, Device, Function: the 16-bit address Windows assigns to a PCIe function during enumeration. It is not in cfg-space; it is in the upstream switch's routing table. Your device's view of its own BDF comes from the Hard IP, which captures it from configuration write transactions arriving with the function's RID.

6.2 The transient-zero problem

The Xilinx 7-Series PCIe Hard IP exposes the current bus number on a signal (typically `cfg_bus_number` in the wrapper) that reflects what the IP thinks the host has assigned. This signal has a known and important property: under several transient events it momentarily returns to 0 before the IP re-captures the assigned value.

Events that cause a momentary `cfg_bus_number = 0`:

- Hot reset (link partner asserts the in-band reset sequence)
- ASPM L1 exit (link returns from low-power state)
- Function-Level Reset (FLR) if the function supports it
- PME (Power Management Event) cycle
- Some PERST# release sequences if the link comes back faster than the IP's bus number capture

6.3 Why this matters

Two consequences flow from a momentary BDF = 0:

1. **DMA fault on an enforcing IOMMU.** If a DMA request goes out with a BDF the IOMMU doesn't have a mapping for (because the IOMMU has BDF 06:00.0 mapped, not 00:00.0), the request is faulted and logged. You see this as HAL event 15 entries in the Windows system log.
2. **External probes see jitter.** Anything reading the device's own view of its identity (its BDF, via a register exposed to user mode, or via Hard IP introspection through your firmware) sees the BDF value flicker. Real silicon does not behave this way; on a real chip the BDF is captured by registers downstream of the link layer and stays stable through these events.

6.4 The latched BDF technique

Inside the cfg-space module of your firmware, sample `cfg_bus_number` and the device/function numbers once at link-up. Latch them sticky. Any read from your firmware for "current BDF" returns the latched value, never the live signal.

Pseudo-structure (illustrative, not a working module):

```
// At link-up, capture the first non-zero bus number into a sticky register.
// Subsequent transients to 0 do not overwrite it.
always_ff @(posedge clk_pcie) begin
    if (perst_n_sync == 0) begin
        bdf_latched_valid <= 1'b0;
        bdf_latched       <= 16'h0000;
    end else if (!bdf_latched_valid && cfg_bus_number != 8'h00) begin
        bdf_latched_valid <= 1'b1;
        bdf_latched       <= { cfg_bus_number, cfg_device_number, cfg_function_number };
    end
end
```

This is a small structural fix with no timing cost. It silences the IOMMU faults and removes the BDF flicker from external visibility.

6.5 The heartbeat tell

Some legacy DMA firmware tries to keep a host driver “convinced” the device is alive by firing periodic synthetic MSIs at a high rate (1 Hz or faster) regardless of whether the emulated device class would generate interrupts at that rate.

This is a textbook tell. A real storage controller, for example, fires interrupts only when there is work to complete: command done, error, hot-plug event. During genuine idle, the interrupt cadence is zero or sub-hertz. A device that emits an MSI every second during true idle does not match any real chip’s behavior.

Two corrections, both required:

- **Drastically slow any synthetic interrupt.** If a synthetic heartbeat is needed at all (some user-mode DMA control planes rely on one for connection liveness), set the period in the tens of seconds. 50 seconds is a reasonable default for a SATA-class emulation.
- **Gate the heartbeat on real device traffic.** Watch the actual interrupt-emit signal from the class-specific engine (e.g. the AHCI completion interrupt). Each real-traffic interrupt resets the heartbeat counter. During a busy session the synthetic heartbeat never fires; during true idle it fires at its long period, which matches the kinds of background events a real device might generate (link state notifications, etc.).

7. MSI Behavior

Match the donor's interrupt model exactly. Don't advertise capabilities the real chip doesn't have.

7.1 Legacy MSI vs MSI-X

Match the donor. If the real chip exposes only legacy MSI (single-vector or multi-vector via the Multiple Message Capable field), your firmware exposes only legacy MSI. Do not add MSI-X just because the Hard IP supports it.

Legacy MSI capability structure, 32-bit Message Address variant (10 bytes, without Per-Vector Masking):

Offset	Size	Field
+0x00	1 B	Cap ID = 0x05
+0x01	1 B	Next Pointer
+0x02	2 B	Message Control: 64-bit capable bit, MMC, MME, per-vector masking
+0x04	4 B	Message Address (low 32)
+0x08	2 B	Message Data (16-bit)

64-bit Message Address variant (14 bytes, without Per-Vector Masking):

Offset	Size	Field
+0x00	1 B	Cap ID = 0x05
+0x01	1 B	Next Pointer
+0x02	2 B	Message Control (64-bit capable bit = 1)
+0x04	4 B	Message Address (low 32)
+0x08	4 B	Message Address (high 32)
+0x0C	2 B	Message Data (16-bit)

The host writes Message Address and Message Data at enumeration. Your firmware reads them back when emitting an MSI.

7.2 The cfg_interrupt interface

The 7-Series PCIe Hard IP exposes a simple interface for issuing interrupts:

- `cfg_interrupt`, assert to request an interrupt emit.
- `cfg_interrupt_rdy`, pulses when the IP accepts the request.
- `cfg_interrupt_assert`, legacy INTx assert/deassert (unused for MSI).
- `cfg_interrupt_di` / `do`, vector number for multi-vector MSI.

You raise `cfg_interrupt`, wait for `cfg_interrupt_rdy`, drop the request. The IP handles the MWr TLP to the host's MSI address with the correct data payload.

7.3 Multi-source arbitration

When more than one engine can emit an interrupt (e.g. the class-specific completion engine plus a synthetic heartbeat plus a watchdog) you need a fair arbiter. The simplest correct pattern is a round-robin grant with priority bypass for high-priority sources:

```
// Pseudocode shape; do not paste into a design.
sources  = [class_engine_int, watchdog_int, heartbeat_int];
priority = [HIGH,             HIGH,         LOW];
// On each clock: pick the highest-priority asserted source, drive cfg_interrupt.
// On cfg_interrupt_rdy: clear that source's pending flag.
```

What you do not want is multiple sources directly driving `cfg_interrupt`, that is a multi-driver bug (see Chapter 13).

7.4 INTx fallback

Some hosts disable MSI for specific devices via the Command register's "Interrupt Disable" bit (and via MSI Enable in the cap control). If your firmware ever ends up with MSI disabled and no INTx fallback, the host driver will stall waiting for interrupts. For 1:1 emulation, match the donor's behavior: if the real chip falls back to INTx when MSI is disabled, your firmware must too.

8. IOMMU Realities

The IOMMU does not block real silicon from doing DMA. It blocks DMA from BDFs that don't have a mapping. Firmware can't bypass an enforcing IOMMU; what it can do is avoid producing the BDF transients that look like cheating.

8.1 What an IOMMU does

An I/O Memory Management Unit sits between PCIe devices and host memory. It translates device-issued addresses (IOVAs) to host physical addresses through page tables managed by the OS. When configured to enforce, it blocks any DMA from a BDF that does not have a valid mapping for the address being accessed. On modern x86 platforms:

- **Intel:** VT-d, advertised via the DMAR ACPI table.
- **AMD:** AMD-Vi, advertised via the IVRS ACPI table. Single IOMMU engine typically at BDF 00:00.2 on consumer platforms.

8.2 The myth and the reality

The persistent myth in DMA communities is that "good firmware bypasses IOMMU". This is false. An active, enforcing IOMMU blocks DMA on the basis of the source BDF and target IOVA. The device cannot lie about its source BDF; the upstream switch and root complex stamp the RID into every TLP. There is no firmware-side technique that makes an enforcing IOMMU permit unmapped DMA from a non-allowed BDF.

What is actually happening on systems where DMA works "despite" IOMMU presence: the IOMMU is present but not enforcing on the device's BDF. That can be true for several reasons:

1. The OS has not asked the IOMMU to enforce, because no hypervisor / VBS / Kernel DMA Protection layer is active to drive the policy.
2. The OS driver for the device class has been configured to opt out of DMA remapping via a registry override.
3. The BIOS has IOMMU enabled at the firmware level but no policy is set up that involves your card's BDF.

8.3 The Windows enforcement stack

For Windows to actively remap DMA on a PCIe device, all of the following must be in place:

1. BIOS-level IOMMU enabled.
2. A hypervisor running (Hyper-V launched at boot, via `bcdedit hypervisorlaunchtype`).
3. Virtualization-Based Security (VBS) enabled and running.
4. Kernel DMA Protection enabled.
5. The relevant class driver not opted out via `HKLM\SYSTEM\CurrentControlSet\Services\<driver>\Parameters\DmaRemappingCompatible = 0` or per-instance `DMA Management\RemappingSupported = 0`.

On a typical home desktop with no virtualization features installed, items 2–4 are off and item 5 is often off via vendor INF defaults. DMA flows freely on such a system because the OS never asks the IOMMU to enforce. The IOMMU is "present but unused" for that BDF.

8.4 The hidden enabler: WSL2 and friends

Several Windows features quietly install the VirtualMachinePlatform optional feature, which sets `bcdedit hypervisorlaunchtype = Auto`. Once that is set, Hyper-V starts at boot and the enforcement stack activates. The user does not see a dialog or prompt; they just installed:

- WSL2 (or upgraded WSL1 to WSL2)
- Docker Desktop
- Windows Sandbox
- Microsoft Defender Application Guard
- Windows Subsystem for Android
- Hyper-V itself

On a previously-working DMA setup, the regression appears as suddenly failing reads with HAL event 15 IOMMU faults logged. The user has not changed firmware or driver; they enabled VBS by installing one of the above.

8.5 The recovery path

To return to a non-enforcing state on Windows:

```
bcdedit /set hypervisorlaunchtype off
shutdown /s /t 0     :: full power-off, NOT restart
```

Then power on. Verify with:

```
Get-CimInstance Win32_ComputerSystem | Select HypervisorPresent
Get-CimInstance Win32_DeviceGuard -Namespace root\Microsoft\Windows\DeviceGuard `
    | Select VirtualizationBasedSecurityStatus, SecurityServicesRunning
```

Expected: `HypervisorPresent = False`, `VirtualizationBasedSecurityStatus = 0`, `SecurityServicesRunning = {0}`.

> **Restart is not enough.** Hyper-V state survives a restart. You must perform a full shutdown and a separate power-on. Fast Startup can also keep state cached even across a shutdown; disable it under Power Options if you are debugging Hyper-V residue.

8.6 What the firmware contributes

The firmware does not bypass an enforcing IOMMU. What it can do, and what is worth doing, is prevent spurious IOMMU faults that would otherwise occur on link transients:

- Latch the BDF as described in Chapter 6 so DMA requests never go out with BDF = 0.

- Hold DMA emission off during link retraining and PERST# windows.
- Acknowledge MSIs cleanly so the IOMMU completion path doesn’t see orphans.

This is coexistence, not bypass.

9. Storage Class Emulation: AHCI / SATA

The storage stack is the most validating PCIe device class. If your firmware can pass this, simpler classes are a smaller surface to satisfy.

The remaining chapters use AHCI / SATA as a worked example because the storage stack is the most validating class. Everything generalizes; if you can pass the storage stack you can pass less demanding classes.

9.1 What you are emulating

The host sees:

- A PCIe function with Class = 0x010601 (Mass Storage / SATA / AHCI 1.0 programming interface).
- BAR5 mapped as memory, holding the AHCI HBA register file (Generic Host Control + Port registers).
- One or more SATA ports, each with a present, ready disk.
- For each disk: the device responses to ATA commands (IDENTIFY, READ DMA EXT, WRITE DMA EXT, SMART, etc.) that look like a real SATA disk.

9.2 AHCI register file layout

BAR5 (sometimes called ABAR) holds:

Offset	Region
0x000–0x02B	Generic Host Control: CAP, GHC, IS, PI, VS, CCC_CTL, CCC_PORTS, EM_LOC, EM_CTL, CAP2, BOHC
0x100 + N×0x80	Port N registers (CLB, FB, IS, IE, CMD, TFD, SIG, SSTS, SCTL, SERR, SACT, CI, SNTF, FBS, DEVSLP)

9.3 Port-bring-up requirements

For `storahci.sys` to enumerate a disk on a port:

- **PxSSTS** (Port SATA Status, offset 0x128 + N×0x80) must report DET = 3 (device present and PHY ready), SPD = current link speed code, IPM = 1 (active). For Gen3 + active: `0x00000133`.
- **PxSIG** (Port Signature, offset 0x124 + N×0x80) must encode a valid SATA signature. For a SATA disk: `0x00000101`.
- **PxCMD** (Port Command, offset 0x118 + N×0x80) tracks the engine state. ST, FRE, FR, CR bits matter; host writes ST=1 to start, FRE=1 to enable the FIS receive area.
- **PxTFD** (Task File Data) initial value should show the device not busy and ready: typically Status = 0x50 (DRDY | DSC), Error = 0x01. Packed: `0x00000150`.

9.4 COMRESET handling

When the host writes `PxSCTL.DET = 1` (request COMRESET), the spec requires the port to drop to "no device" state until the link comes back. Your firmware must reflect this in PxSSTS:

```
// On read of PxSSTS:
rd_data <= (PxSCTL[3:0] == 4'd1) ? 32'h00000000   // COMRESET in progress
                                 : 32'h00000133;  // present + Gen3 + active
```

Without this, the host's reset sequence loops forever waiting for the SSTS transition.

9.5 Write-1-to-set semantics for PxCI and PxSACT

AHCI 1.3.1 §3.3.14 and §3.3.13: PxCI (Command Issue) and PxSACT (SATA Active) are *write-1-to-set, read-as-cleared-on-completion*. Software writes 1s to set new in-flight bits; software *cannot* clear bits. The HBA clears bits when the command completes.

The naive bug, which a surprising number of donor templates ship with:

```
// WRONG: a write replaces, clobbering in-flight bits the host doesn't know about.
9'h138 : p0_ci <= dwr_data;
```

The correct semantic:

```
// RIGHT: OR-only on host writes.
9'h138 : p0_ci <= p0_ci | dwr_data;
```

The completion path is a separate clear-mask, also OR-style atomic with the host write:

```
// Combined atomic update, prevents same-cycle non-blocking races.
p0_ci <= (p0_ci & ~completion_clear_mask) | host_set_mask;
```

If you leave the naive implementation in place, you get a real and reproducible failure: storahci queues several commands at once (CI = 0x1FFF or similar), the engine completes one, the host queues another by writing CI, the host write replaces the register and clobbers the slots still in flight. Storahci waits forever for completions that will never come, then port-resets, then surprise-removes.

9.6 PxIS and write-1-to-clear

PxIS (Port Interrupt Status) is the inverse: *write-1-to-clear*. The HBA sets bits when interrupt sources fire; software writes 1s to acknowledge and clear specific bits. Symmetric atomic pattern:

```
p0_is <= (p0_is | hba_set_mask) & ~host_clear_mask;
```

Both updates in the same `always_ff` block, computed as combinational mask wires before the clocked assignment.

10. ATA Command Engine Architecture

The HBA shell is the easy part. The ATA command engine that actually serves IDENTIFY, READ, WRITE, FLUSH, SMART is where most of the engineering hours go.

10.1 What the engine does

When the host writes a 1 to `PxCI[N]`, it is telling the HBA "fetch command slot N from the Command List Base (CLB) and execute it". A command slot is 32 bytes containing:

- Command Header: PRDTL (PRDT entry count), CFL (CFIS length), flags (W = write, A = ATAPI, P = prefetchable), PRDBC (PRD Byte Count written by HBA on completion).
- Pointer to the Command Table (in host memory).

The Command Table holds:

- CFIS (Command FIS): the 20-byte Host-to-Device Register FIS encoding the ATA command (READ_DMA_EXT 0x25, WRITE_DMA_EXT 0x35, IDENTIFY 0xEC, SMART 0xB0, etc.) plus LBA, sector count, features.
- PRDT entries: 16-byte descriptors each pointing at a host memory region for data transfer. Multi-PRDT for buffers spanning page boundaries.

10.2 The engine's job

1. Detect CI bit set, fetch the slot.
2. MRd to host memory: fetch Command Header.
3. MRd: fetch Command Table CFIS + PRDT array.
4. Parse the ATA command opcode.
5. Dispatch to a per-opcode handler. Examples:
    - IDENTIFY (0xEC), build the 512-byte IDENTIFY response from a ROM table, send to host via MWrs to the PRDT-pointed buffer.
    - READ DMA EXT (0x25), map the LBA to your storage region, MWr the data to host.
    - WRITE DMA EXT (0x35), MRd the data from host, commit to your storage region.
    - SMART READ DATA (0xB0 / 0xD0), serve the SMART attribute table from ROM.
    - FLUSH CACHE EXT (0xEA), emit the D2H Register FIS, no data.
6. On completion: emit a D2H Register FIS into the FIS receive area, set PRDBC = bytes transferred, clear PxCI[N], set PxIS.DHRS, raise the MSI.

10.3 Main engine vs lite engine

For 75T-class budgets, splitting into two engines is a common pattern:

- **Main engine** handles the initial command burst per ST cycle (the host's enumeration sequence: IDENTIFY, READ LOG EXT, SET FEATURES, READ DMA EXT to LBA 0 for partition probing). Has full PRDT walking, can handle large reads.

- **Lite engine** handles steady-state commands after the initial burst. Often serves from a small set of mapped LBAs (boot sectors, GPT entries, FAT regions, a few content sectors). Limited to short bursts (typically ≤ 8 sectors per command) without PRDT walking.

The split is driven by BRAM budget. PRDT walking requires significant state per command (current PRDT entry index, residual length, current target address). For commands the host issues at high rate during boot, you want the engine to be fast and stateless-ish; for the bulk-data path, you want PRDT walking.

10.4 The cap=8 architectural ceiling without PRDT walking

If your lite engine treats `dba` (the first PRDT entry's base address) as a flat target and increments linearly by 64 bytes per chunk, you are implicitly assuming the host has provided a single contiguous PRDT entry. For reads up to 4096 bytes (= 8 sectors of 512 B), the host typically does provide a single PRDT entry because the buffer fits in one page. Above that, the buffer spans pages, the host splits the PRDT, and your linear-increment assumption writes the second-page payload into the wrong physical address. Kernel memory corruption, BSOD, surprise-remove.

Two ways out:

1. Implement PRDT walking in the lite engine (real engineering work, a few hundred lines of state machine).
2. Route any command with sector count > 8 to the main engine via a dispatcher edit, so only short commands hit the lite engine.

10.5 Default-branch FIS emission

For any ATA command your engine does not have an explicit handler for, the safe behavior is:

- Emit a D2H Register FIS with Status = 0x50 (DRDY | DSC), Error = 0, indicating success with no data.
- Set PRDBC = 0.
- Clear CI bit, set IS.DHRS, raise MSI.

Failing to emit the FIS, even on a no-data command like FLUSH CACHE EXT or CHECK POWER MODE, will cause storahci to time out waiting for completion confirmation, then port-reset.

10.6 Aborting on error

For ATA commands you want to explicitly reject (e.g. an unsupported NCQ opcode if the device doesn't advertise NCQ), emit an ABORT FIS:

- Status = 0x51 (DRDY | ERR | DSC)
- Error = 0x04 (ABRT)

Packed into the FIS DW0: `0x04514034` (byte 0 = FIS type 0x34, byte 1 = 0x40 with the I bit, byte 2 = Status 0x51, byte 3 = Error 0x04). Setting PxIS.TFES (bit 30) and PxTFD = 0x0451 on the completion path tells storahci the slot errored.

11. Disk Image, Filesystem, and the partmgr Wall

This chapter contains the single most insidious failure in the Windows storage stack for FPGA-emulated disks. Internalize it. It will save you weeks.

11.1 The disk image

The disk image is the data your firmware serves in response to READ commands and accepts in response to WRITE commands. It is held in BRAM as a sparse map: certain LBAs are mapped to BRAM slots, all other LBAs return zero. A typical slot map for a small FAT32 disk image is on the order of 12–16 slots × 512 B = 6–8 KB of BRAM.

Slot	LBA	Content
0	0	MBR (partition table)
1	1	GPT primary header (if hybrid)
2	2	GPT partition entries (first sector)
3	DISK_TOTAL−33	GPT backup entries
4	DISK_TOTAL−1	GPT backup header
5	partition_start_LBA	FAT32 BPB
6	partition_start_LBA + 1	FSInfo
7	partition_start_LBA + reserved	FAT first sector
8	data_region_start	Root directory
9	cluster N	Subfolder directory cluster
10	cluster M	File content
11	BPB_LBA + 6	Backup BPB

11.2 fastfat.sys strictness

Windows' `fastfat.sys` validates FAT32 metadata more strictly than the FAT spec suggests. Several invariants:

- **0-byte file MUST have first_cluster = 0.** A 32-byte directory entry with FileSize = 0 but a non-zero FstClusLO/HI will mark the entire containing directory as "corrupted and unreadable". Not just the bad entry, the whole directory.
- **File whose first cluster is "free" in FAT.** If the FAT entry for the file's first cluster is 0 (free), fastfat flags directory corruption.
- **"." and ".." in subdirectories.** The `.` entry points to the subdirectory's own cluster. The `..` entry's first cluster equals the parent's first cluster, except when the parent is the FAT32 root, in which case `..` must be 0, not the root's actual cluster number.

- **Volume label match.** The 11-byte label in the root directory's VOLUME_ID entry must match the BPB's `BS_VolLab` field. Mismatch flags the disk dirty.
- **FSInfo signatures.** Lead signature 0x41615252, structure signature 0x61417272, trailing signature 0xAA550000. Wrong signatures cause fastfat to recompute free clusters by scanning the FAT (slow but safe).
- **Multi-sector directory cluster zero-fill.** After the last valid directory entry, remaining sectors of the directory cluster should read as zeros. Easy to satisfy: leave those LBAs unmapped so the engine's default branch returns zeros.

11.3 The partmgr LBA 2 wall

This is the single most insidious wall in the Windows storage stack for FPGA-emulated disks. Document it. Internalize it.

At driver init, `partmgr.sys`'s `PmReadPartitionAttributesMbr` reads exactly one sector at disk byte offset `BytesPerSector × 2`, that is, disk LBA 2. It interprets the first 16 bytes of that sector as a GUID. If those 16 bytes equal `PARTITION_BASIC_DATA_GUID` (`{EBD0A0A2-B9E5-4433-87C0-68B6B72699C7}`), partmgr reads the next 8 bytes as a GPT-style 64-bit attribute mask:

Bit	Attribute	MSFT_Partition property
60	READ_ONLY	IsReadOnly
61	SHADOW_COPY	IsShadowCopy
62	HIDDEN	IsHidden
63	NO_DRIVE_LETTER	NoDefaultDriveLetter

Partmgr caches the result in a registry value:

```
HKLM\SYSTEM\CurrentControlSet\Enum\SCSI\Disk&Ven_<V>&Prod_<P>\<inst>
    \Device Parameters\Partmgr\AttributesTableCache
```

The cache persists across reboots. On every subsequent boot, `PmInitializeAttributesTableCache` reads from the registry; partmgr does not re-read the disk unless the cache is cleared.

11.4 Why this matters for emulated disks

If your disk image at LBA 2 holds a GPT partition entry whose `PartitionTypeGUID` happens to be `PARTITION_BASIC_DATA_GUID` (which is the GUID for any normal data partition), and whose `PartitionUniqueGuid`'s first 8 bytes (interpreted as a 64-bit LE integer for attribute purposes) happen to have bits 60 or 62 set, then partmgr will set IsReadOnly and IsHidden on your partition. Permanently. Across reboots. With no driver, firmware, or filesystem reason for it.

The fix is mechanical: in your disk image, zero out bytes 16..23 of LBA 2 (the 8 bytes after the PartitionTypeGUID). Partmgr reads attributes = 0, caches 0, your partition stays IsReadOnly = False forever.

> **Diagnosing this from the symptoms alone is nearly impossible.** You will spend days suspecting your IDENTIFY DEVICE bits, your IDENTIFY word 84 / 86 / 87 WWN bits, your MODE SENSE handling, your driver vs storahci. None of those are wrong. The wall is upstream of the storage class. Always check the partmgr registry cache and your disk image's LBA 2 first when IsReadOnly is stuck True.

11.5 Three fixes, in order of permanence

1. **Disk image fix (permanent).** Zero bytes 16..23 of LBA 2 in the disk image. Rebuild firmware. Permanent across all future power cycles.
2. **IOCTL_DISK_SET_PARTITION_ATTRIBUTES_EX (per-power-cycle).** When the firmware can't be rebuilt: issue this IOCTL on the partition device. The IOCTL code is `0x0007C0EC`. The input buffer is 24 bytes: bytes 0..15 hold the `PartitionTypeGUID` (left at zero is fine for this call) and bytes 16..23 hold the 64-bit attribute mask (= 0). Partmgr will write attrs = 0 to disk LBA 2 (silent-ACK'd by the firmware, irrelevant) and update the registry cache. Triggers auto-mount.
3. **Registry surgery (last resort).** Delete `AttributesTableCache`, `PartitionTableCache`, `SnapshotDataCache`, `Attributes` under the disk's Partmgr subkey. Power-cycle. Only useful if the disk image LBA 2 bytes are clean, otherwise partmgr re-reads and re-caches the bad value.

12. Writes and Persistence

If your firmware silently drops writes, the host's next read mismatches what it just wrote. Anything doing write-then-read validation will catch you.

12.1 The PRDBC = sect_count × 512 rule

On every WRITE completion path, the engine must set the PRDBC (PRD Byte Count) field in the Command Header writeback to the number of bytes actually transferred:

```
prdbc_value <= {7'b0, sect_count[15:0], 9'b0};   // = sect_count * 512
```

Skipping this (leaving PRDBC at 0) triggers a real and reproducible failure:

1. The host's WRITE IRP completes with "device returned 0 bytes transferred".
2. Storahci sees `CmdHeader->PRDBC < RequestGetDataTransferLength(Srb)` and calls `RequestSetDataTransferLength(Srb, 0)`.
3. The SRB returns to the cache manager with 0 bytes transferred.
4. `fastfat.sys` logs Event ID 50 with status `0xC000000E (STATUS_NO_SUCH_DEVICE)`.
5. The cache manager retries; same failure; storahci eventually surprise-removes.

If you see Event 50 with status `0xC000000E` after a write, that is the canonical signature of PRDBC = 0 somewhere in the engine's write completion path. Audit every state machine branch that completes a WRITE and confirm PRDBC is set.

12.2 Coalescing buffer for write-back consistency

If your disk image is mostly served from a read-only ROM (BRAM init from disk_image.mem), writes silently dropped at the firmware boundary cause a write-then-read inconsistency: the host writes a sector, reads it back immediately, gets the old ROM content, not the written data. Anything that does write-then-read validation will detect this.

A small write-coalescing buffer fixes it without committing writes to persistent storage. Architecture:

- Fixed array of N "lines", each holding {LBA, 512 bytes of data, valid flag}.
- On WRITE: fetch the sector via MRd from host (one MRd per 64 bytes typical, 8 MRds per sector), pack into a free or LRU line, set the LBA and valid flag.
- On READ: check the coalesce buffer first. If LBA matches a valid line, serve from the buffer. Otherwise serve from ROM (or zero for unmapped LBAs).

16 lines × 1 sector = 8 KB of BRAM. Covers most write-then-read probe patterns without committing to a real persistence mechanism.

12.3 Single-sector write commit to BRAM

If you have BRAM headroom and want writes to actually persist within a session (until power cycle), allocate write-back BRAM regions for the slots that matter: BPB, FSInfo, FAT first sector, root directory

cluster, a few subfolder clusters. Route single-sector writes targeting those slot LBAs to a state machine that:

1. Allocates a 32-beat write into the slot's BRAM.
2. Issues MRds to fetch the 512 bytes from host.
3. Commits to BRAM via the dedicated write port.
4. Sets PRDBC = 512, emits D2H FIS, clears CI.

This is enough to make `format X: /fs:FAT32 /Q /Y` succeed: format writes single sectors to BPB, FSInfo, FAT, root, backup BPB, backup FSInfo. Multi-sector FAT zero-fill goes to the coalesce buffer (or silent-drops on read-back; the unmapped LBAs return zeros, which is a valid empty FAT entry).

12.4 "Protected" slots

Some slots should never be written: the MBR, GPT primary header, GPT entries, GPT backup. Writes targeting those LBAs should silent-ACK with PRDBC = 512 and emit the D2H FIS, but not actually commit to BRAM. This preserves the canonical disk layout across writes; if Windows or a filesystem driver writes to LBA 0, you don't want the MBR rewritten to whatever the host thought was there.

12.5 The throttle

Write paths that issue many back-to-back MRds (to fetch the data from host) can starve other engines in the firmware. A simple per-MRd throttle, e.g. 32 idle cycles between each MRd request, keeps the PCIe TX FIFO from saturating and lets concurrent DMA traffic (from user-mode tools, if any) coexist.

13. SystemVerilog Pitfalls

Two synthesis traps that cost the most time in HDL development: silent multi-driver bugs, and same-cycle non-blocking races. Neither produces a warning. Both produce silent runtime failures.

13.1 The silent multi-driver trap

If a single `reg` or `logic` signal is assigned non-blockingly (`<=`) from two different `always @(posedge clk)` blocks in the same module, Vivado synthesis silently picks one of the two drivers and discards the other. No error. No warning. The "winning" driver becomes the only source.

Example of the trap:

```
always @(posedge clk) begin
    if (bram_addr_valid) bram_rvalid <= 1'b1;
end

always @(posedge clk) begin
    if (rst)
        bram_rvalid <= 1'b0;          // ALSO drives bram_rvalid
    else if (state == DONE)
        bram_rvalid <= 1'b0;
end
```

Vivado will silently choose one block and ignore the other. The signal will be stuck. Often the reset-block driver wins and the signal stays at 0 forever, causing the read path to always emit a fallback or sparse value.

13.2 The discipline

- Use `always_ff` rather than `always @(posedge clk)` . SystemVerilog's `always_ff` enforces single-driver semantics at elaboration time.
- For any signal whose update has cross-cutting concerns (one block handles reset, another handles per-cycle update), merge into a single `always_ff` with explicit reset clauses.
- Grep your codebase periodically for `signal_name <=` across files; more than one location is a yellow flag.

13.3 Same-cycle non-blocking races

Even within a single `always_ff` , if a register has multiple assignments at different positions in the code, Verilog last-write-wins semantics apply. The last textual assignment "wins" when both fire on the same cycle.

```
always_ff @(posedge clk) begin
    // Completion path: clears a bit.
    if (completion) p0_ci <= p0_ci & ~slot_mask;

    // ...lots of code...
```

```
    // Host write path: sets a bit.
    if (host_write_ci) p0_ci <= p0_ci | dwr_data;  // ← This wins on same-cycle race
end
```

If both branches fire in the same cycle, the host-write branch wins using the OLD `p0_ci` value, which still has the bit the completion path was trying to clear. The bit gets re-set. Storahci sees an in-flight command that should be done, busy-waits, DPC watchdog, BSOD 0x133.

13.4 The atomic combined-mask pattern

For any register driven by multiple concerns in one `always_ff`, compute the set and clear masks as combinational wires first, then apply them in a single atomic statement:

```
wire [31:0] ci_set_mask_w   = host_write_ci ? dwr_data : 32'h0;
wire [31:0] ci_clear_mask_w = completion ? slot_mask : 32'h0;
wire        ci_wdog_rescue_w = wdog_timeout;

always_ff @(posedge clk) begin
    if (rst)
        p0_ci <= 32'h0;
    else if (ci_wdog_rescue_w)
        p0_ci <= 32'h0;
    else
        p0_ci <= (p0_ci & ~ci_clear_mask_w) | ci_set_mask_w;
end
```

One non-blocking assignment to `p0_ci` per cycle. No race possible.

13.5 Cross-module diagnostic threading

When debugging, the temptation is to thread a handful of diagnostic signals from a deep submodule up to a register file at the top level. Each thread adds a fan-out point and crosses module boundaries. Across many threads, the routing congestion can make completely unrelated paths slow enough to fail timing or, worse, marginally meet timing but fail under sustained host traffic.

Two rules:

- Keep diagnostic ports inside the submodule they belong to. Add a BAR-mapped register at the submodule's I/O boundary instead of threading individual signals up.
- If you must thread, thread on a single dedicated debug bus (e.g. 32 bits of `dbg_sig`) rather than 10 individual signals.

13.6 The hidden timing cost of meeting timing

A build that closes timing with WNS = +0.020 ns and WHS = +0.020 ns will boot fine on most hardware. What does *not* boot fine is a build that meets timing on paper but introduced cross-module logic with large fan-out, even when the fan-out doesn't appear in the critical path. The Vivado timing report doesn't tell you about this; the only signal is that the design "works in sim" but loads with stutters, surprise-removes, or post-boot instability.

If you've added more than a handful of signals crossing module boundaries between one build and the next, and the new build is unstable, suspect the added threading before suspecting your actual logic change.

14. Vivado Build Reality

Most build issues are cache issues, IP regen issues, or the .Xil directory eating your project. None of them are timing.

14.1 The .Xil directory

Vivado writes per-run temp state to `.Xil/` under the project root. Stale state from a failed run will cause confusing errors on the next attempt. Before re-building after a failure:

```
Remove-Item -Recurse -Force vivado_project
Remove-Item -Recurse -Force .Xil
```

14.2 IP cache wipe before COE changes

When you change a COE file (cfg-space or any other RAM init), the cached synthesized IP output may be served from disk regardless of the new COE. The classic symptom: you change a byte, rebuild, and the resulting `.bin` is byte-identical to the previous build. Fix: wipe the IP gen directory before rebuilding.

```
# For an IP named "bram_pcie_cfgspace":
$ipgen = "vivado_project\<proj>.gen\sources_1\ip\bram_pcie_cfgspace"
$iprun = "vivado_project\<proj>.runs\bram_pcie_cfgspace_synth_1"
Remove-Item -Recurse -Force $ipgen, $iprun -ErrorAction SilentlyContinue
# Then rebuild.
```

14.3 FIFO IP regeneration

Changing FIFO depths or widths in IP-generated FIFOs is fragile. The IP wraps an encrypted core that does parameter validation internally. If you get an "encrypted envelope" error during IP synthesis, the IP's own validation has rejected your parameters with no helpful message. Steps to change a FIFO depth safely:

1. Edit the project's live XCI file at `vivado_project\<proj>.srcs\sources_1\ip\<fifo>\<fifo>.xci`. The source `ip/` copy is not what synthesis reads.
2. Update Input_Depth *and* Output_Depth together. Mismatched values cause `C_RD_DEPTH != C_WR_DEPTH` at synthesis and the IP rejects with "encrypted envelope".
3. Pick the right Fifo_Implementation: Common_Clock_Distributed_RAM caps at 1024; above that switch to Common_Clock_Block_RAM (costs RAMB36s).
4. Keep Programmable_Empty_Type at `Single_Programmable_Empty_Threshold_Constant` if any wrapper references the `prog_empty` port. Removing it from the IP will fail top-level synthesis on a "port does not exist" error.
5. Wipe the IP gen and run dirs manually (the build TCL's `reset_target / generate_target / synth_ip` sequence is not enough).
6. Add an explicit IP regen step in the build TCL before `reset_run synth_1`.

14.4 Timing margins

Worst-case slack thresholds in practice on Artix-7 75T:

- WNS > +0.050 ns: comfortable, expect stable boot.
- WNS +0.020 to +0.050: works on hardware if the design is structurally sound.
- WNS < +0.020 ns or negative: rebuild with a different seed (`-directive` or `-strategy`) or expect post-boot instability under load.
- WHS > +0.020 ns: works on hardware, in practice, despite the conservative guidance to target +0.050.
- WHS < +0.010 ns or negative: rebuild.

What the timing report does *not* capture: routing congestion from many cross-module signals. A build that closes timing comfortably can still be unstable if the design has high fan-out from added diagnostic threading. See Chapter 13.

14.5 Build determinism

Two builds with byte-identical sources and identical Vivado configuration are *not* guaranteed to produce a byte-identical `.bit` . The Place and Route phases use heuristics that may pick different solutions on different runs, especially when IP regen runs from scratch. Do not rely on `.bit` hashes as a “did anything change” check. Instead, hash the constituent sources and IP XCI configurations.

15. Per-Board Identity (DNA Binding)

Two boards with identical firmware bytes look the same to any inventory tool. Mix in per-board entropy from FPGA DNA.

15.1 Why per-board identity matters

If two physical boards produce firmware with byte-identical configuration spaces, IDENTIFY DEVICE responses, and SMART tables, then any inventory of "real" devices that records hardware fingerprints will see them as the same hardware. That's a tell. Real silicon of the same model has different serial numbers, different DSN values (if exposed), different SMART attribute values from differing usage history.

15.2 FPGA silicon DNA

Xilinx 7-Series devices have a per-die 57-bit "Device DNA" value, readable via the DNA_PORT primitive. It is unique per silicon die (with overwhelming probability) and stable across power cycles. It is the canonical source of per-board entropy for FPGA designs.

15.3 Deriving identity

One pattern that works:

1. At build time, query the target board's DNA via JTAG or have the user supply it.
2. Hash the DNA with SHA-256 to a 256-bit output.
3. Take slices of the hash to produce: an ATA serial number (20 ASCII bytes for IDENTIFY words 10..19), a disk signature (4 bytes at MBR offset 0x1B8), a volume serial (4 bytes in the FAT32 BPB), and the GPT partition unique GUIDs.
4. Build per-board `.mem` init files from those derived values.
5. The build artifact for that DNA only works on that DNA's board.

15.4 The runtime alternative

Instead of baking identity at build time, read the DNA at runtime and combine it with constants in registers. This means a single bin works across all boards, but it requires more logic and the DNA values must be available to whatever reads them.

For storage emulation, the build-time approach is usually cleaner: the IDENTIFY ROM and disk image are baked, no runtime computation needed.

15.5 Don't leak too much entropy

If you derive 100% of your identity bytes from DNA, two boards with similar DNA hashes (collision possibilities are vanishingly small, but not zero) could produce overlap. More importantly, deriving everything from DNA gives an attacker who learns the DNA → identity mapping a complete fingerprint. Mix in a per-product constant and a build-time secret so the derivation isn't pure-function-of-DNA.

16. Flashing and Deployment

Flashing without power-cycling does nothing visible. The FPGA loads its bitstream once, at power-on or PROG_B assertion.

16.1 .bit to .bin

Vivado synthesizes a `.bit` file (the bitstream in Xilinx native format). For SPI flash deployment, convert to a `.bin` using a small TCL script:

```
write_cfgmem -format BIN -size 16 -interface SPIx4 \
    -loadbit "up 0x0 <path>\<design>.bit" \
    -file "<output>.bin"
```

16 MB SPI x4 is typical for CaptainDMA-class boards. Adjust to the actual flash size on your board.

16.2 JTAG flashing

RECOMMENDED: NEXUSFORGE

For most users, **NexusForge** is the easiest path. It is a Windows GUI that detects supported CaptainDMA-class boards over JTAG, reads the silicon DNA, bundles the CH347 driver, and writes the `.bin` to SPI flash with verify. Download from github.com/NexusForge-dev/NexusForge. Single self-contained `.exe`, no Python, no OpenOCD config files to hunt down, no driver installer to run separately.

Workflow: plug the board's JTAG into your PC, launch NexusForge, click Detect, accept driver install if prompted, pick your `.bin`, click Flash. Done.

CLI ALTERNATIVE: OPENOCD

If you prefer a scripted toolchain, OpenOCD with a CH347 adapter:

```
openocd -f interface/ch347.cfg -f target/<board>.cfg \
        -c "init; reset halt; flash write_image erase <bin> 0x0 bin; reset run; exit"
```

For FT2232H, swap the interface config to `interface/ftdi/<board-specific>.cfg`.

16.3 Power cycle

Reflashing the SPI does not load the new bitstream. The FPGA loaded its current bitstream at the last PROG_B assertion or power-on. Until you assert PROG_B or power-cycle, the old bitstream is still running. "Flashed" without "power-cycled" is the same as not having flashed.

> **Verify the load.** After power cycle, read shadow cfg-space byte 0x30 (or another byte you specifically changed) through a host-side tool like RWEverything. If the byte reflects the new value, the new

> bitstream loaded. If it reflects the old value, the FPGA did not power-cycle or the flash write didn't take.

16.4 Hot-plug behavior

On a board without a PERST# return path that survives reflashing, the host PCIe slot will see the device drop off the link during PROG_B and may not re-enumerate without a reboot of the host. Some boards have a JTAG-triggered link reset; others need a host reboot. Know your board.

17. Live Verification Probes

After flashing and power-cycling, validate from the host side that the firmware presents correctly. These probes need no special tools.

17.1 PCI enumeration

```
Get-PnpDevice -PresentOnly | Where-Object { $_.InstanceId -like 'PCI\VEN_<VID>*' } | ForEach-Objec
    Write-Host "FriendlyName: $($_.FriendlyName)"
    Write-Host "InstanceId:   $($_.InstanceId)"
    Write-Host "Status:       $($_.Status)"
    $hw = (Get-PnpDeviceProperty -InstanceId $_.InstanceId -KeyName 'DEVPKEY_Device_HardwareIds')
    Write-Host "HardwareIds:  $($hw -join ' | ')"
}
```

Verify VID/DID/SVID/SSID/REV match the donor exactly.

17.2 Driver binding

```
Get-CimInstance Win32_PnPSignedDriver | Where-Object { $_.DeviceID -like '*VEN_<VID>*' } |
    Format-List DeviceID, DriverVersion, DriverProviderName, InfName, Manufacturer, Signer
```

If a vendor INF is installed, you should see the vendor's signed driver. If Windows fell back to the inbox class driver, the vendor INF wasn't installed or didn't match the IDs, functional, but less realistic.

17.3 PCI link state

```
$iid = (Get-PnpDevice -PresentOnly | Where-Object { $_.InstanceId -like 'PCI\VEN_<VID>*' }).Instar
foreach ($k in 'DEVPKEY_PciDevice_CurrentLinkSpeed','DEVPKEY_PciDevice_CurrentLinkWidth',
               'DEVPKEY_PciDevice_AcsSupport','DEVPKEY_PciDevice_AtsSupport','DEVPKEY_PciDevice_Sr
    $v = (Get-PnpDeviceProperty -InstanceId $iid -KeyName $k -ErrorAction SilentlyContinue).Data
    Write-Host ("{0,-50} : {1}" -f $k, $v)
}
```

Verify CurrentLinkSpeed and CurrentLinkWidth match the donor (e.g. Gen2 x1 = 2 and 1). AcsSupport, AtsSupport, SriovSupport should match the real chip's capability table.

17.4 Storage class probes (if emulating storage)

```
Get-PhysicalDisk | Where-Object { $_.BusType -eq 'SATA' } | Format-List *
Get-Disk | Format-List Number, PartitionStyle, IsReadOnly, IsOffline, UniqueId
Get-Partition | Format-List DiskNumber, PartitionNumber, DriveLetter, Type, IsHidden, IsActive
Get-Volume | Format-List DriveLetter, FileSystem, FileSystemLabel, HealthStatus
```

Match Model, SerialNumber, FirmwareVersion, Size (in LBAs and bytes), MediaType. Verify IsReadOnly = False, IsHidden = False, drive letter assigned.

17.5 SMART pass-through

```
Get-CimInstance -Namespace root\wmi -ClassName MSStorageDriver_FailurePredictStatus -ErrorAction S
    Format-List InstanceName, PredictFailure, Active
$sm = Get-CimInstance -Namespace root\wmi -ClassName MSStorageDriver_FailurePredictData -ErrorActi
if ($sm) { ($sm.VendorSpecific[0..29] | ForEach-Object { '{0:X2}' -f $_ }) -join ' ' }
Get-PhysicalDisk | Get-StorageReliabilityCounter | Format-List PowerOnHours, Temperature, Wear, Re
```

Verify SMART attribute structure matches the real chip's layout. PowerOnHours non-zero (used-disk signal). Temperature varies with sustained traffic if your firmware models temperature drift.

17.6 Round-trip write

```
$payload = "probe $(Get-Date -Format 'o') $([guid]::NewGuid())"
Set-Content X:\probe.txt $payload -Encoding ASCII
$readback = Get-Content X:\probe.txt -Raw
if ($readback.Trim() -eq $payload) { "WRITE+READ OK" } else { "MISMATCH" }
Remove-Item X:\probe.txt -Force
```

Confirms the full path: AHCI WRITE DMA EXT to disk, AHCI READ DMA EXT back, host MRd/MWr both directions.

17.7 Same-handle repeated reads

A subtle class of firmware bug is a write-cache or buffer pointer that gets reset between reads. Catch it with a same-handle read consistency check:

```
$stream = [System.IO.File]::Open('\\.\PhysicalDriveN', 'Open', 'Read', 'ReadWrite')
$buf = New-Object byte[] 4096
$prev = $null
for ($i = 0; $i -lt 10; $i++) {
    $stream.Position = 0
    [void]$stream.Read($buf, 0, 4096)
    $h = [System.BitConverter]::ToString($buf[0..63]) -join ''
    if ($prev -and $prev -ne $h) { Write-Host "READ $i DIFFERS"; break }
    $prev = $h
}
$stream.Close()
```

Ten consecutive reads through the same handle should return byte-identical data. If they don't, the firmware has stateful reads it shouldn't have.

17.8 What user-mode probes cannot reach

- Raw cfg-space byte dump (need RWEverything or a kernel-mode driver).

- BAR MMIO with custom register reads (same).
- Read latency bypassing the OS file cache (need `FILE_FLAG_NO_BUFFERING` + native I/O).
- Direct PCIe TLP traces (need a protocol analyzer).

For the user-mode-visible surface, the probes above cover what most host-side validation will look at.

Appendix A. Common Errors and Diagnostics

A symptom-driven table of the failures you will hit and what they actually mean.

A.1 “Error generated from encrypted envelope”

Vivado IP synthesis. The IP’s internal parameter validator rejected your inputs without specific reasoning. Most common causes: FIFO Input_Depth != Output_Depth, FIFO depth exceeds the implementation’s max for the chosen RAM type, a removed port that the wrapper still references. See Chapter 14.

A.2 BSOD 0x139 KERNEL_SECURITY_CHECK_FAILURE

The kernel detected an out-of-bounds buffer write or stack corruption. In the context of storage emulation, almost always means the engine wrote DMA data to a wrong host address. The classic cause is multi-PRDT confusion: the engine treated a multi-PRDT-entry buffer as flat and wrote the second-page payload into the wrong physical page.

Fix: implement PRDT walking, or cap sector counts so the lite engine never sees multi-PRDT buffers.

A.3 BSOD 0x133 DPC_WATCHDOG_VIOLATION

A DPC ran longer than the watchdog allowed (default around several seconds, varies by Windows version; cumulative ISR/DPC timeout is larger). In storage emulation: storahci spinning waiting for an in-flight command to complete. Most common cause is the same-cycle non-blocking race on PxCI described in Chapter 13: the host’s CI write clobbers a slot the completion path was trying to clear, the host queues a new command in the same slot index, storahci sees an impossible state and busy-waits.

Fix: atomic combined-mask pattern for PxCI / PxIS / PxSACT updates.

A.4 HAL Event 15 IOMMU faults

Event log: `System / HAL / Event 15`, with Property[0] = the offending BDF and Property[1] = the IOVA. Means the IOMMU is enforcing on a BDF it doesn’t have a mapping for. Causes:

- BDF = 0 transients from the PCIe Hard IP (see Chapter 6). Fix: latched BDF.
- An enforcing IOMMU (Hyper-V + VBS + KDP running) that doesn’t have an opt-out for your card’s driver. Fix: turn off the enforcement stack or add the registry overrides. See Chapter 8.

A.5 fastfat Event ID 50

“Delayed write failed” with status code in the event data. Extract the status:

```
$xml = "<QueryList><Query Id='0' Path='System'><Select Path='System'>
*[System[Provider[@Name='fastfat'] and EventID=50]]</Select></Query></QueryList>"
$evt = Get-WinEvent -FilterXml $xml -MaxEvents 1
'{0:X8}' -f [BitConverter]::ToUInt32($evt.Properties[2].Value, 20)
```

Status	Meaning	Likely cause
0xC000000E	STATUS_NO_SUCH_DEVICE	PRDBC = 0 on a write completion (see Chapter 12)
0xC0000102	STATUS_FILE_CORRUPT_ERROR	Filesystem validation rejected the file or directory; check fastfat strictness rules (Chapter 11)
0xC0000185	STATUS_IO_DEVICE_ERROR	Storahci surprise-removed mid-IRP

The popup that may accompany Event 50 ("Windows, Delayed Write Failed : Exception Processing Message 0xc0000222") is misleading. `0xc0000222` is the popup mechanism's own status, not the actual disk error. Decode from the event body, not the popup.

A.6 Disk vanishes from Device Manager on rescan

Disk was enumerated but disappears when the user clicks "Scan for hardware changes" or when Disk Management triggers a rescan. The PCIe Hard IP itself has wedged (gone fatal) and dropped off the bus. Cold boot reconfigures the FPGA from SPI; warm reboot may or may not recover depending on PERST# behavior.

Suspect causes: a state machine in the firmware wedged in a TX-wait state with no recovery path; the AHCI engine sent a malformed TLP that exhausted the IP's credit pool; PxSACT or PxCI semantics violated under load.

Add watchdog timers to TX-wait states with conservative timeouts (1 ms is too aggressive; tens of ms is safer). On timeout, reset the engine's transaction state and complete the in-flight command with an error status.

A.7 IsReadOnly stuck True on the partition

Almost always the partmgr LBA 2 cache. Fix per Chapter 11. Less commonly, the disk is genuinely write-protected at the device level (real disks can be); check IDENTIFY word 85 bit 12 and the Removable Media Bit if it matters for your emulation.

A.8 Build produces byte-identical .bin despite source changes

Vivado is serving cached IP outputs. Wipe the IP gen + run directories and rebuild. See Chapter 14.

Appendix B. Reading List

PCIE

- PCI Express Base Specification, latest revision: TLP anatomy, configuration space, capability structures, link layer.
- Xilinx PG054: 7-Series Integrated Block for PCI Express, Hard IP user guide.
- Xilinx UG482: 7-Series GTX/GTH transceivers (for the PHY layer of the PCIe block).

SATA / AHCI

- Serial ATA AHCI Specification Revision 1.3.1: register semantics, command flow, FIS formats.
- ATA/ATAPI Command Set (ACS) specification: ATA opcodes, IDENTIFY DEVICE layout, SMART subcommands.

WINDOWS STORAGE STACK

- Microsoft public documentation on the storage driver stack (partmgr, volmgr, mountmgr, fastfat).
- Microsoft public PDBs from the symbol server: `https://msdl.microsoft.com/download/symbols/`, essential for any serious investigation of partmgr, volmgr, fastfat behavior. Set `_NT_SYMBOL_PATH` and use the Microsoft symbol server with WinDbg or Ghidra/IDA Pro.

VIVADO

- UG901 Synthesis, UG904 Implementation, UG903 Timing Constraints, UG908 Programming and Debugging.
- The 7-Series PCIe Integrated Block product guide PG054, again, for IP customization details.

STARTER CODEBASES (OPEN SOURCE)

- **pcileech-fpga**: github.com/ufrisk/pcileech-fpga, the de facto Artix-7 PCIe HDL base, recommended starter for first builds (see Chapter 2.5).
- **PCILeechFWGenerator**: github.com/voltcyclone/PCILeechFWGenerator, Python tooling that generates cfg-space and writemask COEs from a captured donor.
- **verilog-pcie**: github.com/alexforencich/verilog-pcie, Alex Forencich's modern AXI-Stream PCIe IP collection.
- **litepcie** (LiteX): github.com/enjoy-digital/litepcie, Migen-based PCIe IP.
- **corundum**: github.com/corundum/corundum, NIC-class designs.
- **taxi, verilog-axi**: more reference HDL for PCIe-attached engines.
- **cocotbext-pcie, cocotbext-axi**: simulation tooling.
- **Xilinx dma_ip_drivers** (XDMA, QDMA): reference designs for high-throughput DMA patterns.

IOMMU

- Intel VT-d specification: DMAR, IRTE, page-table walk.

- AMD I/O Virtualization Technology specification: IVRS, IOMMU page tables.
- Microsoft documentation on Kernel DMA Protection and DmaRemappingCompatible registry policies.

Appendix C. Glossary

Plain-language definitions of terms used throughout this guide. Listed alphabetically.

AHCI
: Advanced Host Controller Interface. Intel-defined standard for how a SATA host bus adapter (HBA) exposes itself to the OS. AHCI 1.3.1 is the spec version most current chips conform to.

ATA / ATAPI
: Advanced Technology Attachment / Packet Interface. The command set spoken between the host and a SATA disk. IDENTIFY DEVICE (0xEC), READ DMA EXT (0x25), WRITE DMA EXT (0x35), SMART (0xB0), FLUSH CACHE EXT (0xEA) are the most common opcodes.

BAR
: Base Address Register. A 32 or 64-bit register in PCIe configuration space that tells the host where in physical address space the device's MMIO or I/O region should be mapped. AHCI controllers expose BAR5 (ABAR) for the HBA register file.

BDF
: Bus, Device, Function. The 16-bit address Windows assigns to a PCIe function during enumeration. Typically written as `06:00.0` meaning bus 6, device 0, function 0.

BRAM
: Block RAM. On-chip memory inside an FPGA. Artix-7 75T has 105 RAMB36 blocks for a total of 3.8 Mb.

CFIS
: Command FIS. The 20-byte Host-to-Device Register FIS that encodes one ATA command (opcode, LBA, sector count, features).

cfg-space
: Configuration space. 4 KiB of registers per PCIe function holding identity (VID/DID), class code, BAR descriptors, capabilities, link state, and power management. The host reads this during enumeration.

COE
: Coefficient file. A Vivado RAM initialisation file format that pre-populates BRAM contents at synthesis time. Used to bake the donor's configuration-space contents into the FPGA bitstream.

COMRESET
: SATA link-layer reset signal. Host writes `PxSCTL.DET = 1` to request COMRESET; the device must drop link state and rebuild.

DMA
: Direct Memory Access. A device reading from or writing to host memory directly, without CPU involvement, by issuing PCIe TLP transactions.

DMAR
: DMA Remapping. The ACPI table on Intel platforms that describes the IOMMU configuration to the OS.

DNA
: Device DNA. A 57-bit per-die unique identifier in Xilinx 7-Series FPGAs, readable via the DNA_PORT primitive. Used as per-board entropy for identity-binding firmware.

Donor

The real, physical PCIe device whose identity and behaviour your firmware emulates. See Chapter 3.

DSN

Device Serial Number. An optional PCIe extended capability (ID 0x0003) that exposes a 64-bit per-device serial number. Many low-end chips do not expose it.

FIS

Frame Information Structure. The SATA link-layer message format. Several FIS types exist: Host-to-Device Register (0x27), Device-to-Host Register (0x34), DMA Setup (0x41), Data (0x46), etc.

GHC

Generic Host Control. The first ~0x2C bytes of the AHCI BAR5 register file (CAP, GHC, IS, PI, VS, etc.).

HBA

Host Bus Adapter. The chip that bridges PCIe to SATA. The thing you are emulating when you write AHCI firmware.

Hard IP

A fixed-function silicon block inside the FPGA, distinct from the user logic fabric. The 7-Series Integrated Block for PCI Express is hard IP that implements the PCIe physical, data link, and transaction layers.

IOMMU

Input/Output Memory Management Unit. Translates device-issued addresses (IOVAs) to host physical addresses and can block unmapped DMA when enforcing. Chapter 8.

IOVA

I/O Virtual Address. The address a device puts on the wire in a TLP. The IOMMU translates this to a host physical address.

IVRS

I/O Virtualisation Reporting Structure. The ACPI table on AMD platforms equivalent to DMAR on Intel, describing the IOMMU layout.

LBA

Logical Block Address. The sector number passed to a SATA disk. 28-bit LBA reaches up to 128 GiB; 48-bit LBA reaches up to 128 PiB. Modern disks use 48-bit LBA via the EXT commands (READ DMA EXT, WRITE DMA EXT).

MAX_PATH

The legacy 260-character path length limit on Windows. Causes silent Vivado build failures when project paths run deep.

MMIO

Memory-Mapped I/O. Device registers exposed at a physical address range so the CPU can read/write them with normal load/store instructions.

MRd / MWr

Memory Read / Memory Write. The two PCIe TLP types your firmware uses to read from and write to host memory.

MSI / MSI-X

Message Signalled Interrupts. The device delivers interrupts by issuing a Memory Write TLP to a host-configured address with a host-configured data value, instead of asserting a wire.

PERST#

PCI Express Reset. A platform-level reset signal asserted by the host. Active-low.

PRDT

Physical Region Descriptor Table. The list of host memory regions a command transfers to or from. One entry = one 4 KiB-or-less host buffer.

PRDBC

PRD Byte Count. The field in the AHCI Command Header that the HBA writes on completion to indicate how many bytes were actually transferred.

PxCI

Port Command Issue. The 32-bit register at AHCI port offset 0x38 where each bit corresponds to a command slot. Host writes 1 to set; HBA writes 0 to clear on completion.

PxIS

Port Interrupt Status. The 32-bit register at AHCI port offset 0x10 where the HBA sets bits to indicate interrupt sources. Host writes 1 to clear.

PxSACT

Port SATA Active. The 32-bit register at AHCI port offset 0x34 used for NCQ command tracking.

RAMB36

The unit of block RAM in Xilinx 7-Series FPGAs. 36 Kib each (configurable as 32 Kib of pure data plus parity).

RID

Requester ID. The 16-bit BDF value stamped into every PCIe TLP by the source function. Used by the IOMMU for source-based mapping decisions.

SMART

Self-Monitoring, Analysis, and Reporting Technology. ATA command opcode 0xB0 with subcommands for reading the disk's health attributes, error logs, and self-test status.

TLP

Transaction Layer Packet. The unit of PCIe communication: header + optional data payload. Memory Read, Memory Write, Configuration Read, Configuration Write, Completion, and Message are the main types.

VBS

Virtualization-Based Security. A Windows feature that uses Hyper-V to isolate sensitive code. When enabled, it activates the IOMMU enforcement stack.

VID / DID

Vendor ID / Device ID. The 16-bit identifiers at cfg-space offset 0x00 and 0x02 that Windows uses to bind drivers.

VT-d

Intel Virtualization Technology for Directed I/O. Intel's name for their IOMMU. The AMD equivalent is AMD-Vi.

WHS / WNS

Worst Hold Slack / Worst Negative Slack. The Vivado timing report metrics that tell you whether your design met setup and hold timing constraints. Negative values mean the design will fail at the target clock speed.

Join the community

Questions, build problems, share your work, contribute fixes to the guide: join the Nexus Discord.

discord.gg/y49jPf8T5z

NexusForge releases, future revisions of this guide, and discussion of FPGA emulation engineering happen there.

End of guide. Published by Nexus for the community. Vendor-agnostic. No proprietary code. No anti-cheat specifics. Provided as engineering reference.